

Actionable Runbook: Fixing the 3 Docker Traps

This is a **hands-on, step-by-step execution guide**. Follow these exact terminal commands and file edits to permanently resolve the networking, volume, and database initialization issues in your `loan-tracker` project.

Trap 1: The "Localhost" CORS / Proxy Bug

Symptom: The React app crashes or fails to load data when accessed from another device (like a phone) because it is hardcoded to look for `http://localhost:8000`.

Step-by-Step Fix

1 Edit the React API Configuration

Open your frontend API utility file. We need to remove the hardcoded localhost URL so Nginx can handle the routing.

 frontend/src/utils/api.js

```
import axios from 'axios';

const api = axios.create({
  -   baseURL: 'http://localhost:8000/api', // <-- REMOVE THIS
  +   baseURL: '/api', // <-- ADD THIS
  headers: { 'Content-Type': 'application/json' }
});
```

2 Rebuild the Frontend Container

Since React code is bundled, you must rebuild the container to apply this change.

```
tony@Jarvis:~/docker/loan-tracker$ docker-compose build frontend
tony@Jarvis:~/docker/loan-tracker$ docker-compose up -d frontend
```

3 Verify the Fix

How to test: Open the app on your smartphone (using your laptop's IP address, e.g., `http://192.168.1.X:8090`). If the dashboard loads data successfully, the trap is fixed!

Trap 2: The Bind Mount Override (Dependency Crash)

Symptom: When moving to a new laptop or deploying to production, running `docker-compose up` crashes the backend immediately with `ModuleNotFoundError`. Your host folder is erasing the container's built-in Python libraries.

Step-by-Step Fix (For Production / Moving Systems)

Note: Only do this when you are ready to deploy or move systems. If you are actively writing code right now, keep bind mounts ON so your code live-reloads.

1 Disable Volumes in Docker-Compose

We must tell Docker to use the files baked into the image, NOT the files on your laptop.

 **docker-compose.yml**

```
backend:

  build: ./backend

  ports:

    - "8000:8000"

- volumes:                                # <-- COMMENT THIS OUT

-   - ./backend:/app                      # <-- COMMENT THIS OUT

+   # volumes:                            # <-- SHOULD LOOK LIKE THIS

+   #   - ./backend:/app                  # <-- SHOULD LOOK LIKE THIS

  depends_on:

    - db
```

2 Force a Clean Rebuild

Now, build a fresh image and start the container without the volume override.

```
# 1. Stop the current backend tony@Jarvis:~/docker/loan-tracker$ docker-
compose stop backend # 2. Rebuild the image from scratch (ensures pip
install runs) tony@Jarvis:~/docker/loan-tracker$ docker-compose build --no-
cache backend # 3. Start it up tony@Jarvis:~/docker/loan-tracker$ docker-
compose up -d backend
```

3 Verify the Fix

How to test: Run `docker logs loan_tracker_backend`. If it says "Application startup complete" and does NOT say "ModuleNotFoundError", the container is successfully running independently of your host laptop.

Trap 3: The `init.sql` Ignored Update Bug

Symptom: You added a new column (like `updated_at`) to `backend/migrations/init.sql`, but the database ignores it and the backend crashes with a missing column error.

Step-by-Step Fix (The "Backup & Nuke" Method)

Since Postgres only reads `init.sql` when the hard drive is empty, we will use the custom scripts you built to save your data, empty the drive, and put the data back.

1 Generate a Fresh JSON Backup

Save all your current loans and targets to a JSON file.

```
tony@Jarvis:~/docker/loan-tracker$ bash scripts/backup.sh # Verify a new
file appeared in the backups/ folder! tony@Jarvis:~/docker/loan-tracker$ ls
-l backups/
```

2 Destroy the Database Volume

Bring down the entire stack and permanently delete the Postgres volume.

```
# The -v flag is critical! It deletes the volume. tony@Jarvis:~/docker/loan-
tracker$ docker-compose down -v
```

3 Restart to Trigger `init.sql`

Bring the stack back up. Because the volume was destroyed, Postgres will execute your modified `init.sql`.

```
tony@Jarvis:~/docker/loan-tracker$ docker-compose up -d # Wait 10 seconds
for Postgres to finish initializing...
```

4 Restore Your Data

Now, run your Python restore script *inside* the backend container to insert your JSON data into the newly created tables.

```
# Execute the restore script inside the running container tony@Jarvis:~/  
docker/loan-tracker$ docker exec -it loan_tracker_backend python app/  
restore.py
```

5 Verify the Fix

How to test: Open your React app and try to use the feature that was crashing before (e.g., saving a target). If it succeeds, the database successfully built the new columns and restored your old data!